PROXMOX VE 3-NODE CLUSTER

Complete Command Execution & Operations Runbook (Start to Finish)

CLUSTER NODES	IP ADDRESSES	STORAGE STACK	ENVIRONMENT
prox22, prox23, prox24	10.31.6.172 / 173 / 174	Ceph 20.2 + ZFS RAID-Z2	PVE 9.2 (Debian 13 Linux)

1. Verified Physical Storage Topology & Bay Layout

All three Dell PowerEdge servers feature an identical front 12-bay chassis layout mapped through Dell PERC H755 / MegaRAID controller:

Chassis Bay	Role	Drive Specs	prox22 (10.31.6.172)	prox23 (10.31.6.173)	prox24 (10.31.6.174)
Slot 0	CEPH OSD	18.2TB Seagate HDD	osd.0 (S/N: ZYDGFEW6)	osd.1 (S/N: ZYDFR75F)	osd.2 (S/N: ZYDFWT0C)
Slot 1	CEPH OSD	18.2TB Seagate HDD	osd.3 (S/N: ZYDG4WXP)	osd.4 (S/N: ZYDG3L2Q)	osd.5 (S/N: ZYDFMNJ2)
Slot 2	CEPH OSD	18.2TB Seagate HDD	osd.6 (S/N: ZYDG0CL3)	osd.7 (S/N: ZYDGCX8R)	osd.8 (S/N: ZYDEHZW4)
Slot 3	CEPH OSD	18.2TB Seagate HDD	osd.9 (S/N: ZYDFZ35V)	osd.10 (S/N: ZYDGD4MM)	osd.11 (S/N: ZYDEML2L)
Slots 4–10	ZFS RAID-Z2	7 × 18.2TB HDDs (zfs-ha)	7 Drives (ZYDG62WJ ... ZYDFWL9E)	7 Drives (ZYDG1DK6 ... ZYDDT9JV)	7 Drives (ZYDG9GLB ... ZYDG9GR2)
Slot 11	SPARE	18.2TB Standby Drive	S/N: ZYDFW4AP	S/N: ZYDG7ZYQ	S/N: ZYDFHHAE
Rear Bay	OS ROOT	960GB Micron SSD	/dev/sdm (pve-root)	/dev/sdm (pve-root)	/dev/sdm (pve-root)

2. Phase 1 — Hardware Discovery & Physical Topology Mapping

Commands executed to establish cluster network connectivity, inspect RAID controllers, and map Linux block devices to physical backplane slot numbers:

2.1 Network Connectivity & SSH Reachability Tests

BASH — LOCAL WORKSTATION

```
# 1. Ping and reachability test across all 3 nodes
for ip in 10.31.6.172 10.31.6.173 10.31.6.174; do
    ping -c 1 -W 1 $ip 2>&1 | grep "packets transmitted"
done

# 2. Port 22 SSH verification
for ip in 10.31.6.172 10.31.6.173 10.31.6.174; do
    nc -z -v -w 1 $ip 22 2>&1
done

# 3. Passwordless SSH verification & Hostname checks
for ip in 10.31.6.172 10.31.6.173 10.31.6.174; do
    ssh -o ConnectTimeout=2 -o BatchMode=yes root@$ip "hostname"
done
```

2.2 Storage Controller & Physical Drive Slot Discovery (PERCCLI64)

SSH — prox22 (10.31.6.172)

```
# Locate Dell PERC CLI utility on Proxmox node
which perccli /opt/MegaRAID/perccli/perccli64 storcli 2>/dev/null

# Inspect Dell PERC Controller 0 backplane enclosure and slots
/opt/MegaRAID/perccli/perccli64 /c0 /eall /sall show

# Query drive slots and serial numbers on Enclosure 252
/opt/MegaRAID/perccli/perccli64 /c0 /e252 /sall show all | grep -E "Drive /c0/e252/s|SN ="
```

2.3 Ceph Cluster & OSD Storage Mapping

SSH — prox22 (10.31.6.172)

```
# View Ceph OSD tree, hosts, and weights
ceph osd tree

# Correlate Ceph OSD daemon IDs to device paths and drive serial numbers
ceph device ls

# Extract OSD IDs and serials for automated mapping
ceph device ls 2>/dev/null | awk "NR>1 {print \$2, \$3}"
```

2.4 ZFS Pool ('zfs-ha') Drive Mapping

SSH — ALL NODES (prox22, prox23, prox24)

```
# Check ZFS pools across the cluster
for ip in 10.31.6.172 10.31.6.173 10.31.6.174; do
    echo "=== $ip ==="
    ssh root@$ip "zpool list; zfs list -o name,mountpoint 2>/dev/null"
done

# Query physical disk members of 'zfs-ha' pool with persistent paths
ssh root@10.31.6.172 "zpool status -P -v zfs-ha"
ssh root@10.31.6.173 "zpool status -P -v zfs-ha"
ssh root@10.31.6.174 "zpool status -P -v zfs-ha"
```

3. Phase 2 — Physical Hard Drive Finder & LED Blinker Deployment

Developed and deployed customized automation tools to help technicians identify physical drive caddies and blink server LEDs before applying physical stickers.

3.1 Deploying Finder Tooling to /output/ Across All 3 Nodes

BASH / SCP — DEPLOY TO CLUSTER

```
# Create /output directories and deploy sticker finder scripts & dashboard
for ip in 10.31.6.172 10.31.6.173 10.31.6.174; do
    echo "=== Deploying to $ip ==="
    ssh root@$ip "mkdir -p /output /output/bin"
    scp harddrive-sticker-finder.sh root@$ip:/output/harddrive-sticker-finder.sh
    scp cluster_drive_stickers_viewer.html root@$ip:/output/cluster_drive_stickers_viewer.html
    ssh root@$ip "chmod +x /output/harddrive-sticker-finder.sh && ls -lh /output"
done
```

3.2 Running Hardware Scan on Cluster Nodes

SSH — RUN ON EACH NODE

```
# Scan local server hardware (Outputs table with Slot, Role, Device, S/N, Model)
/output/harddrive-sticker-finder.sh --scan

# Remote scan execution from workstation over SSH
ssh root@10.31.6.172 '/output/harddrive-sticker-finder.sh --scan'
ssh root@10.31.6.173 '/output/harddrive-sticker-finder.sh --scan'
ssh root@10.31.6.174 '/output/harddrive-sticker-finder.sh --scan'
```

3.3 Physical Drive Locator LED Blinking via Dell PERC

SSH — PHYSICAL IDENTIFICATION

```
# Blink drive caddy activity/locate LED by Serial Number for 15 seconds
/output/harddrive-sticker-finder.sh --blink ZYDGFEW6 15

# Low-level Dell PERCCLI commands used under the hood to start/stop locate:
/opt/MegaRAID/perccli/perccli64 /c0/e252/s0 start locate
sleep 15
/opt/MegaRAID/perccli/perccli64 /c0/e252/s0 stop locate
```

4. Phase 3 — Prometheus Exporters Full Installation (All 3 Nodes)

Step-by-step commands used to deploy, configure, and activate all 5 monitoring exporters across `prox22`, `prox23`, and `prox24`.

4.1 Exporter 1: Node Exporter (Port 9100) — OS, CPU, RAM & Disk IO

BASH — ALL 3 NODES

```
# 1. Install package via apt
apt-get update && apt-get install -y prometheus-node-exporter

# 2. Enable and start systemd service
systemctl enable --now prometheus-node-exporter

# 3. Verify metrics endpoint
curl -s http://localhost:9100/metrics | grep -E "node_cpu_seconds_total|node_memory_MemTotal_bytes" | hea
d -n 5
```

4.2 Exporter 2: IPMI Exporter (Port 9290) — Dell Fans, Temps & PSUs

BASH — ALL 3 NODES

```
# 1. Install exporter package and low-level IPMI driver tools
apt-get install -y prometheus-ipmi-exporter freeipmi-tools ipmitool

# 2. Load and persist kernel IPMI modules
modprobe ipmi_si ipmi_devintf
cat << 'EOF' > /etc/modules-load.d/ipmi.conf
ipmi_si
ipmi_devintf
EOF

# 3. Configure local IPMI collection profile in /etc/default/prometheus-ipmi-exporter
cat << 'EOF' > /etc/default/prometheus-ipmi-exporter
ARGS="--config.file=/usr/share/doc/prometheus-ipmi-exporter/examples/ipmi_local.yml"
EOF

# 4. Configure service to execute as root to access /dev/ipmi0
sed -i 's/^User=prometheus/User=root/' /lib/systemd/system/prometheus-ipmi-exporter.service 2>/dev/null |
| true
systemctl daemon-reload
systemctl enable --now prometheus-ipmi-exporter

# 5. Verify sensor metrics output
curl -s http://localhost:9290/metrics | grep -E "ipmi_fan_speed_rpm|ipmi_temperature_celsius" | head -n 5
```

4.3 Exporter 3: Smartctl Exporter (Port 9633) — Drive S.M.A.R.T Health

BASH — ALL 3 NODES

```
# 1. Ensure smartmontools is present
apt-get install -y smartmontools

# 2. Copy pre-built smartctl_exporter binary to system binary path
cp /output/bin/smartctl_exporter /usr/local/bin/smartctl_exporter
chmod 755 /usr/local/bin/smartctl_exporter

# 3. Create systemd service with Dell PERC virtual disk exclusion (--smartctl.device-exclude=sdm)
cat << 'EOF' > /etc/systemd/system/smartctl_exporter.service
[Unit]
Description=Prometheus Smartctl Exporter (Drive Health & Temps)
Documentation=https://github.com/prometheus-community/smartctl_exporter
After=network.target

[Service]
Type=simple
User=root
ExecStart=/usr/local/bin/smartctl_exporter --web.listen-address=:9633 --smartctl.device-exclude=sdm
Restart=always
RestartSec=5s
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
EOF

# 4. Reload systemd, enable, and start
systemctl daemon-reload
systemctl enable --now smartctl_exporter

# 5. Verify drive health metrics
curl -s http://localhost:9633/metrics | grep -E "smartctl_device_temperature|smartctl_device_status" | he
ad -n 5
```

4.4 Exporter 4: Ceph Prometheus Exporter (Port 9283) — Ceph MGR Module

BASH — PROXMOX / CEPH CLUSTER

```
# 1. Enable native Prometheus exporter module inside Ceph Manager
ceph mgr module enable prometheus

# 2. Set listening port and bind address
ceph config set mgr mgr/prometheus/server_port 9283
ceph config set mgr mgr/prometheus/server_addr 0.0.0.0

# 3. Check Ceph Manager status and verify active manager node
ceph mgr stat

# 4. Verify metric emission on the active Ceph manager (prox22)
curl -s http://10.31.6.172:9283/metrics | grep -E "ceph_health_status|ceph_mon_quorum_status" | head -n 5
```

4.5 Exporter 5: ZFS Exporter (Port 9134) — 'zfs-ha' Pool & ARC Stats

BASH — ALL 3 NODES

```
# 1. Copy pre-built zfs_exporter binary to system binary path
cp /output/bin/zfs_exporter /usr/local/bin/zfs_exporter
chmod 755 /usr/local/bin/zfs_exporter

# 2. Create systemd service for ZFS metrics
cat << 'EOF' > /etc/systemd/system/zfs_exporter.service
[Unit]
Description=Prometheus ZFS Exporter (zfs-ha Pool & ARC Stats)
Documentation=https://github.com/pdf/zfs_exporter
After=network.target

[Service]
Type=simple
User=root
ExecStart=/usr/local/bin/zfs_exporter --web.listen-address=:9134
Restart=always
RestartSec=5s

[Install]
WantedBy=multi-user.target
EOF

# 3. Reload systemd, enable, and start
systemctl daemon-reload
systemctl enable --now zfs_exporter

# 4. Verify ZFS metrics
curl -s http://localhost:9134/metrics | grep -E "zfs_pool_health|zfs_arc_hits" | head -n 5
```

5. Phase 4 — Unified Exporter Automation & Health Checker

Developed `/output/install-cluster-exporters.sh` to streamline one-touch deployments and live multi-node status verifications.

5.1 Local Node Health Check

BASH — RUN ON TARGET HOST

```
/output/install-cluster-exporters.sh --status localhost
```

5.2 Entire Cluster Multi-Node Health Check

BASH — RUN FROM ANY NODE

```
/output/install-cluster-exporters.sh --status cluster
```

Live Execution Output on All 3 Nodes:
• **prox22 (10.31.6.172):** Node (6,037 metrics) • IPMI (346 metrics) • Smartctl (733 metrics) • **Ceph (358 metrics - Active MGR)** • ZFS (109 metrics)
• **prox23 (10.31.6.173):** Node (7,769 metrics) • IPMI (346 metrics) • Smartctl (732 metrics) • **Ceph (STANDBY - Active on prox22)** • ZFS (109 metrics)
• **prox24 (10.31.6.174):** Node (7,774 metrics) • IPMI (346 metrics) • Smartctl (732 metrics) • **Ceph (STANDBY - Active on prox22)** • ZFS (109 metrics)

6. Phase 5 — Ceph HA Architecture Diagnosis & Port 9283 Resolution

When running health checks on `prox24`, port 9283 returned 0 metrics. Here are the exact diagnostic commands executed to uncover Ceph's High Availability manager behavior:

6.1 Diagnosing Port 9283 and Ceph Manager State

SSH — CLUSTER DIAGNOSTICS

```
# 1. Check which daemon is listening on port 9283 across all 3 nodes
ssh root@10.31.6.172 "ss -tulpn | grep 9283"
ssh root@10.31.6.173 "ss -tulpn | grep 9283"
ssh root@10.31.6.174 "ss -tulpn | grep 9283"

# 2. Query Ceph Manager active and standby election state
ceph mgr stat
# Output: {"epoch": 66, "available": true, "active_name": "prox22", "num_standby": 2}

# 3. Inspect HTTP response code and payload size on each node
curl -I -s http://10.31.6.172:9283/metrics  # HTTP 200, Content-Length: 32789 (Active)
curl -I -s http://10.31.6.173:9283/metrics  # HTTP 200, Content-Length: 0     (Standby)
curl -I -s http://10.31.6.174:9283/metrics  # HTTP 200, Content-Length: 0     (Standby)

# 4. Check root web endpoint on standby node (prox24)
curl -s http://10.31.6.174:9283/
# Returns HTML pointing to active manager: <a href='http://192.168.10.22:9283/metrics'>
```

Ceph Architectural Rule: Ceph Manager operates in **Active/Standby mode**. Only the currently elected Active MGR (`prox22`) serves cluster-wide metrics. Standby MGRs (`prox23` and `prox24`) intentionally return 0 metrics to prevent Prometheus from collecting duplicate timeseries. If `prox22` reboots, Ceph automatically fails over to `prox23` or `prox24`, which will immediately begin emitting the full metrics.

7. Phase 6 — Removal of Old Standalone Prometheus Server on prox22

An old standalone Prometheus installation (created August 24, 2026) was identified on `prox22` at `/opt/prometheus` listening on port 9090. Since Prometheus runs on a dedicated external server, the local instance was completely decommissioned.

7.1 Inspecting Old Prometheus Files & Service

SSH — prox22 (10.31.6.172)

```
# Check status and file paths
systemctl status prometheus --no-pager
ls -la /opt/prometheus
ls -l /etc/systemd/system/prometheus.service
```

7.2 Decommissioning & Removal Commands

SSH — prox22 (10.31.6.172)

```
# 1. Stop the running Prometheus server process
systemctl stop prometheus

# 2. Disable systemd service from booting automatically
systemctl disable prometheus

# 3. Delete systemd service unit file & reload daemon
rm -f /etc/systemd/system/prometheus.service
systemctl daemon-reload
systemctl reset-failed

# 4. Delete binary and TSDB data directory (~250MB freed)
rm -rf /opt/prometheus
```

7.3 Post-Removal Verification

SSH — prox22 (10.31.6.172)

```
# 1. Verify service is completely gone
systemctl status prometheus --no-pager
# Output: Unit prometheus.service could not be found.

# 2. Verify port 9090 is closed
ss -tulpn | grep 9090
# Output: <empty> (Port closed)

# 3. Verify directory is deleted
ls -ld /opt/prometheus
# Output: ls: cannot access '/opt/prometheus': No such file or directory

# 4. Verify all 5 cluster exporters remain 100% active
/output/install-cluster-exporters.sh --status localhost
```

8. Phase 7 — External Prometheus Server Scrape Configuration

The scrape configuration specification to place in your **dedicated external Prometheus server's** `prometheus.yml` file:

YAML **EXTERNAL PROMETHEUS CONFIGURATION**

```yaml
scrape_configs:
  # 1. Proxmox OS & System Performance (CPU, RAM, Disks, Network)
  - job_name: 'proxmox-nodes'
    scrape_interval: 15s
    static_configs:
      - targets:
          - '10.31.6.172:9100'
          - '10.31.6.173:9100'
          - '10.31.6.174:9100'

  # 2. Dell Server Hardware Sensors (Fans RPM, Temperatures, Power Supplies)
  - job_name: 'proxmox-ipmi'
    scrape_interval: 30s
    scrape_timeout: 20s
    static_configs:
      - targets:
          - '10.31.6.172:9290'
          - '10.31.6.173:9290'
          - '10.31.6.174:9290'

  # 3. Hard Drive S.M.A.R.T Health (Drive Wear, Bad Sectors, Temperatures)
  - job_name: 'proxmox-smartctl'
    scrape_interval: 60s
    scrape_timeout: 45s
    static_configs:
      - targets:
          - '10.31.6.172:9633'
          - '10.31.6.173:9633'
          - '10.31.6.174:9633'

  # 4. Ceph Cluster & OSD Metrics (Scraped from Ceph MGRs with auto failover)
  - job_name: 'ceph-mgr'
    scrape_interval: 15s
    static_configs:
      - targets:
          - '10.31.6.172:9283'
          - '10.31.6.173:9283'
          - '10.31.6.174:9283'

  # 5. ZFS Pool Health & ARC Metrics ('zfs-ha' RAID-Z2 Pool)
  - job_name: 'proxmox-zfs'
    scrape_interval: 30s
    static_configs:
      - targets:
          - '10.31.6.172:9134'
          - '10.31.6.173:9134'
          - '10.31.6.174:9134'
```

9. Master Quick-Reference Command Table

Consolidated summary of essential operations commands across the cluster:

Task / Operation	Command	Target Node
Full Cluster Exporter Health	`/output/install-cluster-exporters.sh --status cluster`	Any Node
Local Exporter Health	`/output/install-cluster-exporters.sh --status localhost`	Any Node
Hard Drive Bay & Role Scan	`/output/harddrive-sticker-finder.sh --scan`	Any Node
Blink Physical Drive LED	`/output/harddrive-sticker-finder.sh --blink <SERIAL> 15`	Target Node
Check Active Ceph MGR	`ceph mgr stat`	Any Node
Check ZFS Pool Status	`zpool status -v zfs-ha`	Any Node
Restart Smartctl Exporter	`systemctl restart smartctl_exporter`	Any Node
Restart IPMI Exporter	`systemctl restart prometheus-ipmi-exporter`	Any Node
Restart ZFS Exporter	`systemctl restart zfs_exporter`	Any Node
Restart Node Exporter	`systemctl restart prometheus-node-exporter`	Any Node
Verify Ceph Metrics	`curl -s http://10.31.6.172:9283/metrics | head -n 10`	Active MGR